

Machine Learning: A Comprehensive Textbook

Solutions to Chapter 11 Exercises

Recurrent Neural Networks and Long Short-Term Memory

Solutions Manual

Exercise 11.1 — BPTT Derivation

For a vanilla RNN with scalar hidden state $h^{(t)} = \tanh(w_{hh}h^{(t-1)} + w_{xh}x^{(t)})$ and loss $L = \sum_t L^{(t)}$, derive $\partial L / \partial w_{hh}$ as a sum over time steps.

Solution. Because w_{hh} is *shared* across all time steps, its total gradient sums the contribution from every time step at which it was used — and crucially, $h^{(t)}$ depends on w_{hh} both *directly* (through the current step’s update) and *indirectly* (through all previous hidden states $h^{(t-1)}, h^{(t-2)}, \dots$, which themselves depend on w_{hh}). Formally, using the total-derivative/multivariable chain rule:

$$\frac{\partial L}{\partial w_{hh}} = \sum_t \frac{\partial L}{\partial h^{(t)}} \cdot \frac{dh^{(t)}}{dw_{hh}},$$

where $\frac{dh^{(t)}}{dw_{hh}}$ denotes the *total* derivative of $h^{(t)}$ with respect to w_{hh} , accounting for the entire chain of dependence back through time. We unroll this recursively: since $h^{(t)} = \tanh(z^{(t)})$ with $z^{(t)} = w_{hh}h^{(t-1)} + w_{xh}x^{(t)}$,

$$\frac{dh^{(t)}}{dw_{hh}} = \tanh'(z^{(t)}) \cdot \left[h^{(t-1)} + w_{hh} \frac{dh^{(t-1)}}{dw_{hh}} \right],$$

(the bracketed term reflects the product rule applied to $z^{(t)} = w_{hh}h^{(t-1)} + w_{xh}x^{(t)}$: the explicit $\partial z^{(t)} / \partial w_{hh} = h^{(t-1)}$ direct term, plus $w_{hh} \cdot dh^{(t-1)} / dw_{hh}$ capturing how w_{hh} also affects $z^{(t)}$ indirectly via its effect on $h^{(t-1)}$).

Unrolling this recursion back to $t = 0$ (with $h^{(0)}$ typically a fixed initial state, so $dh^{(0)} / dw_{hh} = 0$), the standard equivalent form used in practice — Backpropagation Through Time (BPTT) — reorganizes this sum-of-products into a double sum over pairs of time steps:

$$\frac{\partial L}{\partial w_{hh}} = \sum_t \frac{\partial L}{\partial h^{(t)}} \sum_{k=1}^t \left(\frac{\partial h^{(t)}}{\partial h^{(k)}} \right) \frac{\partial h^{(k)}}{\partial w_{hh}} \Big|_{\text{explicit}},$$

where $\frac{\partial h^{(k)}}{\partial w_{hh}} \Big|_{\text{explicit}} = \tanh'(z^{(k)}) h^{(k-1)}$ is the *local, explicit* partial derivative at step k only (treating $h^{(k-1)}$ as a fixed input, not itself a function of w_{hh} — i.e. only the “direct” use of w_{hh} at step k), and

$$\frac{\partial h^{(t)}}{\partial h^{(k)}} = \prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}} = \prod_{j=k+1}^t \tanh'(z^{(j)}) w_{hh}$$

is the Jacobian chain propagating the gradient backward from step t to step k through the intervening hidden states. Putting it together:

$$\frac{\partial L}{\partial w_{hh}} = \sum_t \frac{\partial L}{\partial h^{(t)}} \sum_{k=1}^t \left[\prod_{j=k+1}^t \tanh'(z^{(j)}) w_{hh} \right] \tanh'(z^{(k)}) h^{(k-1)}.$$

This is exactly the BPTT gradient: it sums, over every pair of “current” time step t and “earlier” contributing time step $k \leq t$, the product of (i) the loss sensitivity at t , (ii) the chained Jacobian product propagating the gradient signal backward from t to k , and (iii) the local/explicit gradient at step k . In practice this is computed efficiently backward-in-time (accumulating the running product/recursion rather than the explicit double sum), which is precisely what makes BPTT an $O(T)$ (not $O(T^2)$) algorithm despite the apparent double sum.

Exercise 11.2 — Vanishing Gradient Theorem

Prove that if $|w_{hh}| < 1/(1 - \tanh^2(z_{\max}))$ for all pre-activations, the gradient $\partial h^{(t)}/\partial h^{(k)} \rightarrow 0$ as $t - k \rightarrow \infty$.

Solution. From Exercise 11.1, $\frac{\partial h^{(t)}}{\partial h^{(k)}} = \prod_{j=k+1}^t \tanh'(z^{(j)}) w_{hh}$, a product of $t - k$ terms, each of the form $\tanh'(z^{(j)}) w_{hh}$.

Recall $\tanh'(z) = 1 - \tanh^2(z) \in (0, 1]$, with the maximum value $\tanh'(0) = 1$ attained at $z = 0$, and $\tanh'(z)$ strictly decreasing in $|z|$. The problem statement (as phrased) appears to intend the condition in terms of the *maximum* value $\tanh'$ can attain over the relevant range, i.e. let $M := \max_j \tanh'(z^{(j)}) = 1 - \tanh^2(z_{\min})$ where $z_{\min}$ is whichever pre-activation is closest to 0 (giving the largest possible $\tanh'$ value) — interpreting the exercise’s condition as bounding $|w_{hh}|$ relative to this worst-case (largest) value of $\tanh'$ across all time steps, so that the inequality holds *uniformly* for every factor in the product. Under this reading, suppose

$$|w_{hh}| \cdot \tanh'(z^{(j)}) \leq c < 1 \quad \text{for every } j,$$

for some constant $c < 1$ (this is guaranteed, in particular, if $|w_{hh}| < 1/M$ where $M = \max_j \tanh'(z^{(j)})$ is the largest activation-derivative value occurring along the sequence — matching the spirit of the stated bound with $M = 1 - \tanh^2(z_{\max})$ denoting the derivative evaluated at whichever pre-activation gives the least saturation).

Then each individual factor in the product satisfies $|\tanh'(z^{(j)}) w_{hh}| \leq c < 1$, so

$$\left| \frac{\partial h^{(t)}}{\partial h^{(k)}} \right| = \left| \prod_{j=k+1}^t \tanh'(z^{(j)}) w_{hh} \right| = \prod_{j=k+1}^t |\tanh'(z^{(j)}) w_{hh}| \leq \prod_{j=k+1}^t c = c^{t-k}.$$

Since $0 \leq c < 1$, we have $c^{t-k} \rightarrow 0$ as $t - k \rightarrow \infty$ (a geometric sequence with ratio strictly less than 1 decays to zero). By the squeeze theorem (since $0 \leq |\partial h^{(t)}/\partial h^{(k)}| \leq c^{t-k} \rightarrow 0$):

$$\left| \frac{\partial h^{(t)}}{\partial h^{(k)}} \right| \rightarrow 0 \quad \text{as } t - k \rightarrow \infty. \quad \blacksquare$$

Interpretation. This proves the vanishing gradient problem for vanilla RNNs rigorously: whenever the recurrent weight magnitude $|w_{hh}|$ is small enough relative to the (bounded, ≤ 1) activation derivative $\tanh'$, the gradient signal connecting a loss at time t back to a hidden state at an earlier time k decays *geometrically* (exponentially) in the gap $t - k$ — meaning gradients from far-future losses provide essentially zero learning signal to update the hidden state’s influence at distant past time steps, exactly the practical failure mode that motivated the development of gated architectures like LSTM (Exercise 11.3) and GRU (Exercise 11.5), whose additive/multiplicative-gate cell-state updates are specifically designed to avoid this uniform geometric-decay structure.

Exercise 11.3 — LSTM Cell Gradient

Derive $\partial c^{(t)} / \partial c^{(t-1)} = \text{diag}(f^{(t)})$ from the LSTM cell update equation.

Solution. The LSTM cell-state update is

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)},$$

where $f^{(t)}$ is the forget gate, $i^{(t)}$ the input gate, and $\tilde{c}^{(t)}$ the candidate cell state — all themselves functions of $h^{(t-1)}$ and $x^{(t)}$, but (crucially for this exercise) *not* directly, explicitly functions of $c^{(t-1)}$ (the gates and candidate depend on $c^{(t-1)}$ only *indirectly*, through $h^{(t-1)}$, which is a separate computation from the direct term below — the exercise asks for the *direct/local* partial derivative treating $f^{(t)}, i^{(t)}, \tilde{c}^{(t)}$ as fixed with respect to $c^{(t-1)}$, which is the relevant quantity for the “constant error carousel” argument).

Differentiating the update equation element-wise with respect to $c^{(t-1)}$, treating $f^{(t)}$ as (locally) independent of $c^{(t-1)}$ for this direct-path derivative (i.e. computing $\partial c^{(t)} / \partial c^{(t-1)}$ along the direct additive pathway, which is the term responsible for the LSTM’s signature gradient-preserving behavior):

$$\left. \frac{\partial c^{(t)}}{\partial c^{(t-1)}} \right|_{\text{direct}} = \frac{\partial}{\partial c^{(t-1)}} [f^{(t)} \odot c^{(t-1)}] = \text{diag}(f^{(t)}),$$

since for elementwise (Hadamard) product $f^{(t)} \odot c^{(t-1)}$, the Jacobian with respect to $c^{(t-1)}$ is precisely the diagonal matrix with $f^{(t)}$ ’s entries on the diagonal (each output coordinate $c_j^{(t)}$ ’s direct dependence on $c^{(t-1)}$ is $\partial(f_j^{(t)} c_j^{(t-1)}) / \partial c_j^{(t-1)} = f_j^{(t)}$, and $\partial(f_j^{(t)} c_j^{(t-1)}) / \partial c_k^{(t-1)} = 0$ for $k \neq j$, since coordinate j of the elementwise product depends only on coordinate j of each input). The second term $i^{(t)} \odot \tilde{c}^{(t)}$ contributes zero to this direct derivative, since (by the stated convention of isolating the direct pathway) it does not directly involve $c^{(t-1)}$ as a multiplicative factor. Hence

$$\frac{\partial c^{(t)}}{\partial c^{(t-1)}} = \text{diag}(f^{(t)}). \quad \blacksquare$$

Significance (the “constant error carousel”). Unlike the vanilla RNN’s hidden-state Jacobian (Exercise 11.2), which involves a *multiplication* by the recurrent weight matrix w_{hh} (or W_{hh}) composed with a saturating $\tanh'$ nonlinearity at every step — producing geometric decay unless very specific conditions on $|w_{hh}|$ hold — the LSTM’s cell-state Jacobian is *purely* the diagonal forget-gate matrix $\text{diag}(f^{(t)})$, with *no* additional weight-matrix multiplication or nonlinearity-derivative attenuation baked directly into this pathway. If the forget gate $f^{(t)} \approx 1$ (which the network can learn to achieve, e.g. via the large positive forget-gate bias initialization studied in Exercise 11.6), the cell-state gradient can flow backward through many time steps with *no attenuation at all* (Jacobian $\approx I$), directly analogous to the residual-connection gradient-preservation mechanism of Exercise 10.4 — this is precisely the architectural insight (Hochreiter & Schmidhuber, 1997) that allows LSTMs to learn much longer-range temporal dependencies than vanilla RNNs.

Exercise 11.4 — LSTM Parameter Count

For an LSTM with n_h hidden units and d_x input dimension, count the parameters for each gate. Show the total is $4(n_h^2 + n_h d_x + n_h)$.

Solution. An LSTM has four gates/computations at each time step, each of the form

$$g^{(t)} = \phi(W_g x^{(t)} + U_g h^{(t-1)} + b_g),$$

for $g \in \{f \text{ (forget)}, i \text{ (input)}, o \text{ (output)}, \tilde{c} \text{ (candidate)}\}$, where ϕ is σ for the three gates and $\tanh$ for the candidate. Each gate has its own independent set of parameters:

- $W_g \in \mathbb{R}^{n_h \times d_x}$: maps the input $x^{(t)} \in \mathbb{R}^{d_x}$ to the n_h -dimensional gate pre-activation — $n_h \times d_x$ parameters.
- $U_g \in \mathbb{R}^{n_h \times n_h}$: maps the previous hidden state $h^{(t-1)} \in \mathbb{R}^{n_h}$ — $n_h \times n_h = n_h^2$ parameters.
- $b_g \in \mathbb{R}^{n_h}$: bias — n_h parameters.

So each individual gate has $n_h d_x + n_h^2 + n_h$ parameters. Since there are exactly 4 such gates (forget, input, output, candidate), each with its own independent weight matrices and bias, the total parameter count is

$$\text{Total} = 4(n_h^2 + n_h d_x + n_h). \quad \blacksquare$$

(Equivalently, this can be seen by noting that in a typical vectorized implementation, all four gates' input-to-hidden weights are often concatenated into one matrix $W \in \mathbb{R}^{4n_h \times d_x}$ and all four hidden-to-hidden weights into $U \in \mathbb{R}^{4n_h \times n_h}$, plus a bias $b \in \mathbb{R}^{4n_h}$, giving the identical total count $4n_h d_x + 4n_h^2 + 4n_h = 4(n_h^2 + n_h d_x + n_h)$.) For comparison, a vanilla RNN has only a single such gate-like computation, giving just $n_h^2 + n_h d_x + n_h$ parameters — so an LSTM has exactly 4× as many parameters as a vanilla RNN of the same hidden size, precisely reflecting its four separate gating/candidate mechanisms.

Exercise 11.5 — GRU Reduces to RNN

Show that the GRU reduces to a vanilla RNN when the update gate $z^{(t)} \equiv 1$ and the reset gate $r^{(t)} \equiv 1$.

Solution. The GRU update equations are:

$$\begin{aligned} r^{(t)} &= \sigma(W_r x^{(t)} + U_r h^{(t-1)} + b_r), & z^{(t)} &= \sigma(W_z x^{(t)} + U_z h^{(t-1)} + b_z), \\ \tilde{h}^{(t)} &= \tanh(W_h x^{(t)} + U_h(r^{(t)} \odot h^{(t-1)} + b_h), \\ h^{(t)} &= (1 - z^{(t)}) \odot h^{(t-1)} + z^{(t)} \odot \tilde{h}^{(t)}. \end{aligned}$$

Setting $r^{(t)} \equiv 1$. The candidate-state equation simplifies: $r^{(t)} \odot h^{(t-1)} = 1 \odot h^{(t-1)} = h^{(t-1)}$ (the reset gate no longer suppresses any part of the previous hidden state before it enters the candidate computation), so

$$\tilde{h}^{(t)} = \tanh(W_h x^{(t)} + U_h h^{(t-1)} + b_h),$$

which is exactly the vanilla RNN's hidden-state update formula (matching, e.g., Exercise 11.1's $h^{(t)} = \tanh(w_{hh} h^{(t-1)} + w_{xh} x^{(t)})$ in vector form, with U_h playing the role of w_{hh} and W_h the role of w_{xh}).

Setting $z^{(t)} \equiv 1$. The final hidden-state update simplifies:

$$h^{(t)} = (1 - 1) \odot h^{(t-1)} + 1 \odot \tilde{h}^{(t)} = 0 + \tilde{h}^{(t)} = \tilde{h}^{(t)}.$$

So the update gate no longer performs any interpolation/blending between the old hidden state and the new candidate — the hidden state is simply replaced entirely by the candidate at every step, exactly as in a vanilla RNN (which has no gating mechanism at all — the hidden state is unconditionally overwritten by the new computation each step).

Combining both. With $r^{(t)} \equiv 1$ and $z^{(t)} \equiv 1$:

$$h^{(t)} = \tilde{h}^{(t)} = \tanh(W_h x^{(t)} + U_h h^{(t-1)} + b_h),$$

which is exactly the vanilla RNN recurrence. ■ This confirms that the GRU is a strict architectural generalization of the vanilla RNN: setting both gates to their “fully open”/“always update” configuration recovers the plain RNN exactly as a special case, and the gates’ role is precisely to let the network *learn*, per-timestep and per-dimension, how much of the past hidden state to retain (via z) and how much of it to expose to the candidate computation (via r) — a strictly more expressive and (as with LSTM’s forget gate) gradient-flow-friendly generalization of the plain recurrence.

Exercise 11.6 — Forget Gate Initialisation

Explain why initialising the forget gate bias b_f to a large positive value (e.g., 1 or 5) helps gradient flow in early training. Compute $f^{(t)}$ for $b_f = 0, 1, 5$ assuming other inputs are zero.

Solution. **Computing $f^{(t)}$.** With $f^{(t)} = \sigma(W_f x^{(t)} + U_f h^{(t-1)} + b_f)$ and all other inputs (i.e. $W_f x^{(t)} + U_f h^{(t-1)}$) set to zero, $f^{(t)} = \sigma(b_f)$:

$$b_f = 0 : \quad f^{(t)} = \sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5.$$

$$b_f = 1 : \quad f^{(t)} = \sigma(1) = \frac{1}{1 + e^{-1}} = \frac{1}{1 + 0.3679} \approx \frac{1}{1.3679} \approx 0.731.$$

$$b_f = 5 : \quad f^{(t)} = \sigma(5) = \frac{1}{1 + e^{-5}} = \frac{1}{1 + 0.006738} \approx \frac{1}{1.006738} \approx 0.9933.$$

Why a large positive bias helps early training. From Exercise 11.3, the cell-state gradient across one time step is $\partial c^{(t)} / \partial c^{(t-1)} = \text{diag}(f^{(t)})$ — the forget gate directly controls how much of the backpropagated gradient signal survives each step. At initialization (before the network has learned anything task-specific), if $b_f = 0$, the forget gate starts at $f^{(t)} = 0.5$: this means, on average, *half* the cell-state gradient/information is discarded at *every single time step*, so across a sequence of length T , the surviving gradient/information fraction decays roughly as 0.5^T — an extremely fast (still geometric, though less severe than the tanh-based vanilla-RNN decay) attenuation that reintroduces a version of the vanishing-gradient problem the LSTM architecture was specifically designed to avoid, especially damaging in the earliest phase of training, before the network has had a chance to learn (via gradient updates driven by exactly this attenuated signal) that certain gates should be kept more open.

Initializing b_f to a large positive value (e.g. 1 or 5) instead starts the forget gate close to 1 ($f^{(t)} \approx 0.731$ or ≈ 0.993 respectively, as computed above) — meaning the cell state is, by default, nearly fully *preserved* across time steps at the very start of training (Jacobian $\text{diag}(f^{(t)}) \approx I$, mirroring the “identity-like” gradient-preserving behavior of residual connections, Exercise 10.4). This gives the network long-range memory and healthy gradient flow *from the very first training iterations*, rather than requiring the optimizer to first discover — purely from a heavily-vanishing gradient signal — that the forget gate should be opened up before it can even begin learning longer-range dependencies. This is a well-known and widely-adopted practical initialization trick (Jozefowicz et al. 2015, among others), and many deep learning framework LSTM implementations (e.g. Keras) apply exactly this large-positive-forget-bias initialization by default.